

Modelos Generativos Profundos

Clase 5: Modelos autorregresivos y LLMs (parte I)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Formulación de un LM

Inferencia usando una RNN

Formulación de un LM

Vocabulario

Si $\mathcal{V}_0 = \{a_1, \dots, a_K\}$ es un vocabulario común, los elementos de una secuencia $S = (w_1, \dots, w_T) \in \mathcal{V}^*$ se denominan **tokens**.

Es habitual considerar un vocabulario extendido, $\mathcal{V} \supset \mathcal{V}_0$, para incluir **tokens especiales**. Por ejemplo:

- **Begin of sentence:** <BOS>.
- **End of sentence:** <EOS>.
- **Unknown:** <UNK>.
- **Padding:** <PAD>.

En este caso:

$$\mathcal{V} = \{a_1, \dots, a_K, \underbrace{\text{<BOS>}}_{a_{K+1}}, \underbrace{\text{<EOS>}}_{a_{K+2}}, \underbrace{\text{<UNK>}}_{a_{K+3}}, \underbrace{\text{<PAD>}}_{a_{K+4}}\}.$$

Corpus y conjunto de entrenamiento

- Un **modelo de lenguaje** es un modelo generativo que asigna una probabilidad a cada secuencia de tokens $S \in \mathcal{V}^*$, es decir, aprende una distribución $p_\theta : \mathcal{V}^* \rightarrow [0, 1]$.
- Para entrenar un LM, es necesario contar con un conjunto de entrenamiento $\mathcal{D} = \{S^1, \dots, S^N\} \subset \mathcal{V}^*$.
- Dado un **corpus** con símbolos en $\mathcal{V}$, se puede construir $\mathcal{D} \subset \mathcal{V}^*$ considerando todas las subsecuencias que se puedan obtener a partir de dicho corpus.
- Las secuencias en $\mathcal{D}$ se suelen truncar o extender (usando <PAD>) a un mismo largo $T \in \mathbb{N}$. Esto permite entrenamiento en batches.

Formulación autorregresiva

Dada una secuencia de tokens, $S = (w_1, \dots, w_T) \in \mathcal{V}^*$, un **modelo autorregresivo** descompone la probabilidad conjunta $p_\theta(S)$ de manera causal:

$$p_\theta(w_1, \dots, w_T) = p_\theta(w_1) \prod_{t=1}^{T-1} p_\theta(w_{t+1} | w_1, \dots, w_t)$$

- w_1 siempre es <BOS>, por lo que se puede fijar $p_\theta(w_1) = \delta_{\text{<BOS>}}(w_1)$.
- Por lo tanto, solo se requiere aprender $p_\theta(w_{t+1} | w_1, \dots, w_t)$ para todo contexto $(w_1, \dots, w_t) \in \mathcal{V}^*$ y posible continuación $w_{t+1} \in \mathcal{V}$.
- Con esta distribución aprendida, es posible generar nuevas secuencias de tokens mediante **ancestral sampling**.

Formulación autorregresiva

- $p_{\theta}(w_{t+1}|w_1, \dots, w_t) \sim \text{Categ\acute{o}rica}(\mathcal{V}; \lambda_{\theta}(w_1, \dots, w_t))$ es una distribución sobre el vocabulario $\mathcal{V}$, es decir:

$$p_{\theta}(w_{t+1} = a_k | w_1, \dots, w_t) = \lambda_{\theta}(w_1, \dots, w_t)_k \in [0, 1]$$

- El vector de probabilidades $\lambda_{\theta}(w_1, \dots, w_t) \in \Delta^{|\mathcal{V}|}$ es aprendido por una red neuronal $\lambda_{\theta} : \mathcal{V}^* \rightarrow [0, 1]^{|\mathcal{V}|}$.
- Esta red neuronal recibe un **contexto** $(w_1, \dots, w_t) \in \mathcal{V}^*$ y entrega una distribución para el próximo token, $w_{t+1} \in \mathcal{V}$.

Inferencia usando una RNN

Vectores de embedding

- Las redes neuronales esperan como entrada vectores de $\mathbb{R}^D$, por lo que es necesario obtener una **representación continua** para cada uno de los símbolos del vocabulario $\mathcal{V}$.
- Se debe asociar cada token $a \in \mathcal{V}$ con un **vector de embedding** $\text{emb}(a) \in \mathbb{R}^D$.
- El operador $\text{emb} : \mathcal{V} \rightarrow \mathbb{R}^D$ mapea el k -ésimo token de $\mathcal{V}$ a la k -ésima fila de una matriz $E \in \mathcal{M}_{|\mathcal{V}|, D}(\mathbb{R})$.
- La matriz E se llama **matriz de embedding** y es aprendida junto al resto de los parámetros de la red neuronal.

Redes neuronales recurrentes

- Dado que la entrada $(w_1, \dots, w_t) \in \mathcal{V}^*$ a la red neuronal es de largo variable, no es posible utilizar una red MLP estándar para parametrizar la distribución $p_\theta(w_{t+1}|w_1, \dots, w_t)$.
- Una alternativa clásica para este tipo de problemas es usar una **red neuronal recurrente**. La red de Elman clásica procesa una secuencia de tokens $(w_1, \dots, w_t) \in \mathcal{V}^*$ de forma recursiva:

$$\begin{cases} h_0 = 0 \\ h_s = \tanh(A \text{emb}(w_s) + B h_{s-1} + d), \quad 1 \leq s \leq t \end{cases}$$

- De este modo, se puede considerar que

$$\lambda_\theta(w_1, \dots, w_t) = \text{softmax}(C h_t + e) \in \Delta^{|\mathcal{V}|}$$

Entrenamiento

- La red neuronal se puede entrenar maximizando la verosimilitud (normalizada) sobre un trainset $\mathcal{D} = \{S^1, \dots, S^N\} \subset \mathcal{V}^*$:

$$\max_{\theta} \frac{1}{|\mathcal{D}|} \sum_{S \in \mathcal{D}} \frac{1}{|S|} \log p_{\theta}(S).$$

- Las normalizaciones entregan independencia en el tamaño de batch y en los largos de las secuencias.
- Para una secuencia $S = (w_1, \dots, w_T) \in \mathcal{V}^*$:

$$\begin{aligned} \log p_{\theta}(S) &= \log \left(p_{\theta}(w_1) \prod_{t=1}^{T-1} p_{\theta}(w_{t+1} | w_1, \dots, w_t) \right) \\ &= \sum_{t=1}^{T-1} \log p_{\theta}(w_{t+1} | w_1, \dots, w_t). \end{aligned}$$

Se puede generar una nueva secuencia de tokens utilizando **ancestral sampling** en un LM entrenado:

- Se comienza con el token inicial $w_1 = \text{<BOS>}$ y se evalúa el modelo neuronal para predecir el siguiente token.
- Esto se repite de manera iterativa usando como contexto la secuencia de tokens que ya han sido generados.
- La generación continúa hasta que se genera el token <EOS> o hasta que se genera una cantidad máxima de tokens.
- Se puede realizar **generación condicional** comenzando con una secuencia de tokens iniciales (**prompt**), $S_{\text{prompt}} = (w_1, \dots, w_t)$.

Heurísticas de generación

Se debe decidir cómo elegir el próximo token a partir de $p_{\theta}(w_{t+1}|w_1, \dots, w_t) \sim \text{Categórica}(\mathcal{V}; \lambda_{\theta}(w_1, \dots, w_t))$:

- **Sampling:** elegir w_{t+1} sampleando desde $p_{\theta}(w_{t+1}|w_1, \dots, w_t)$.
- **Greedy decoding:** elegir w_{t+1} como el token más probable, i.e.,

$$w_{t+1} = a_k, \quad \text{donde} \quad k = \arg \max_{k \in \{1, \dots, |\mathcal{V}|\}} \lambda_{\theta}(w_1, \dots, w_t)_k.$$

- **Top-K:** si $\mathcal{I} = \{k_1, \dots, k_K\}$ son las $K \in \mathbb{N}$ coordenadas de mayor valor del vector $\lambda_{\theta}(w_1, \dots, w_t) \in \Delta^{|\mathcal{V}|}$, samplear usando

$$\tilde{\lambda}_{\theta}(w_1, \dots, w_t)_k = \begin{cases} 0 & \text{si } k \notin \mathcal{I} \\ \frac{\lambda_{\theta}(w_1, \dots, w_t)_k}{\sum_{i \in \mathcal{I}} \lambda_{\theta}(w_1, \dots, w_t)_i} & \text{si } k \in \mathcal{I} \end{cases}.$$

Heurísticas de generación

Si $l \in \mathbb{R}^{|\mathcal{V}|}$ es el vector de logits entregado por la red neuronal y $T > 0$ es un parámetro de **temperatura**, se puede escalar el vector de logits $l \mapsto \frac{1}{T}l$ antes de aplicar la función softmax.

El parámetro T permite controlar el trade-off entre versatilidad y confianza en la generación:

- Si $T > 1$, el vector de probabilidades se vuelve más uniforme.
- Si $T < 1$, el vector de probabilidades se vuelve más determinista.

En particular, $T \rightarrow 0$ equivale a greedy decoding, mientras $T \rightarrow \infty$ equivale a un sampling uniforme sobre $\mathcal{V}$.

Heurísticas de generación

Muchas veces la generación autorregresiva puede caer en ciclos de repetición. Para mitigar este problema, se puede aplicar una heurística de **penalización por repetición**.

Si $c(a_k) \in \mathbb{N}$ es la cantidad de veces que el símbolo $a_k \in \mathcal{V}$ ha aparecido en la secuencia generada hasta el momento, se pueden escalar los logits $l \in \mathbb{R}^{|\mathcal{V}|}$ mediante

$$\tilde{l}_k = \begin{cases} \frac{1}{\gamma^{c(a_k)}} \cdot l_k & \text{si } l_k > 0 \\ \gamma^{c(a_k)} \cdot l_k & \text{si } l_k \leq 0 \end{cases}$$

El hiperparámetro $\gamma \geq 1$ controla el nivel de penalización.

Si $\gamma > 1$, entonces los logits de tokens repetidos decrecen, reduciendo su probabilidad tras aplicar softmax.

En la próxima clase:

- Arquitectura GPT.
- 1º Control de lectura.

Modelos Generativos Profundos

Clase 5: Modelos autorregresivos y LLMs (parte I)